The Sicher C Compiler

Runs the following 5 core Modules sequentially...

Sicher Modules

Sub Modules

1

Source

Tokeniser

Takes the input C source file and validates it's textual legibility, ensures legal identifier name, closed blocks of code, the correct string and integer literal formatting et cetera.

- Remove meta characters
- Check for semicolons integrity
- Ensure literal(s) values are correct
- Raise error for any unknown characters encountered

→ Testing Token Suite → Release Testing Suite

2

Source

Parser

Validates the given source programs syntax against the grammar Rules of the C language, checking for bracket scoping, provding ample compiler debug messaging to aid in the parsing.

- Ensure proper syntax
- Ensure correct block scoping
- Raise errors if incorrect syntax in the source file

→ Testing Syntax Suite → Release Syntax Suite

3

AST

Analyser

From the lexical & syntaxically correct program, generate an abstract syntax tree (AST) from it, introduce compile time Countermeasures and optimisations.

- Create a AST from the parsed source code
- Compile-time countermeasures
- Compile-time Optimisations

→ Type Checking → Scope Managing

4

Binary

Code Generator

Generate assembly instructions from the AST, build the executable binary and perform runtime optimisations and countermeasures.

- Apply assembly instructions to the AST
- From the Assembly instructions create a executable binary
- Runtime countermeasures
- Runtime optimisations

→ Common Subexpression Elimination → Consant Folding

5

GUI

Visualiser

Create a GUI for the user to explore how each module worked, with all known-GUI messaging to the user, and devise a top level visual describing the countermeasures & optimisations Sicher took for this source program.

- Show the GUI
- From each of the previous modules, show the module compiler steps to
- Create a top level visual of the given source code along with Sicher's
- Allow for execution of the Sicher compiled binary in the GUI
- File system browsing for next compilable target

→ GUI → CLI

Any of the modules can have sub-modules that extends the functionality of any of the 5 parent module(s), such as adding additional compile-time optimisations in the Analyser Module, or additional runtime ones in the Code Generator Module.